

hello

Assumptions

- This is the user's own Android device.
- The user has USB debugging enabled and adb installed on the computer.
- The app is installed and the user knows its package name (e.g., com.example.vulnerable).
- If the package name is unknown, we include a command to list installed packages.

Commands / Code

1. Find the package name (if unknown)

bash

adb shell pm list packages | grep <keyword

Replace <keyword with a term related to the app (e.g., 'bank', 'chat', 'vuln')

2. Pull the APK to your computer

bash

adb shell pm path <packagename

Example output: package:/data/app/com.example.vulnerable-xxx/base.apk

adb pull /data/app/com.example.vulnerable-xxx/base.apk ./vulnerableapp.apk

3. Basic static analysis (file info, strings, hashes)

bash

file vulnerableapp.apk

sha256sum vulnerableapp.apk

strings vulnerableapp.apk | head -100 look for interesting strings

4. Decompile with apktool (resources, smali)

bash

apktool d vulnerableapp.apk -o vulnerableappsmali

5. Decompile to Java with jadx (readable code)

bash

jadx -d vulnerableappjava vulnerableapp.apk

Or GUI: jadx-gui vulnerableapp.apk

6. Quick manifest check

bash

aapt dump badging vulnerableapp.apk | grep -E 'package:|launchable-activity|permission'

What each command does

- pm list packages – lists all installed packages; filter with grep.
- pm path <package – returns the path of the APK on the device.
- adb pull – copies the APK file from device to your computer.
- file, sha256sum – identify file type and compute hash for integrity.
- strings – extract printable strings (API calls, URLs, hardcoded secrets).

- apktool d – decodes resources and decompiles to smali (Dalvik bytecode).
- jadx -d – decompiles DEX to Java source code for review.
- aapt dump badging – prints manifest info: package name, permissions, activities.

Next safe steps

1. Replace <keyword or <packagename with the actual app identifier.
2. Run the commands in order. If adb devices shows no device, check USB debugging.
3. Use jadx-gui to browse the decompiled code interactively.
4. Look for insecure permissions, hardcoded credentials, exposed components, or weak cryptography.
5. If you find a vulnerability, create a report – do not exploit it outside a lab/CTF environment.